



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

NGHIÊN CỨU VÀ ỨNG DỤNG GIAO THỨC ĐIỀU PHỐI PHI TẬP TRUNG CHO MLS TRÊN MẠNG NGANG HÀNG

Đồ án tốt nghiệp

Người thực hiện:

Lê Tiến Dũng – 20224837

Trường CNTT&TT – Đại học Bách khoa Hà Nội

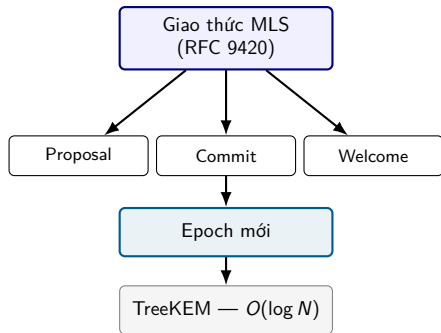
Giảng viên hướng dẫn: TS. Đỗ Bá Lâm

ONE LOVE. ONE FUTURE.

1. Giới thiệu bài toán
2. Giao thức điều phối đề xuất
3. Triển khai & Thực nghiệm
4. Kết luận

Giới thiệu bài toán

Nền tảng: Giao thức MLS (RFC 9420)



Epoch: trạng thái nhóm sau mỗi Commit.

Commit: gom Proposal → epoch mới. **Không giao hoán.**

TreeKEM: cập nhật khóa $O(\log N)$.

FS: thành viên bị loại không đọc tin sau.

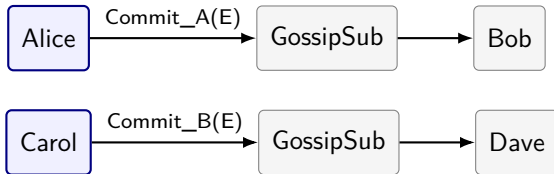
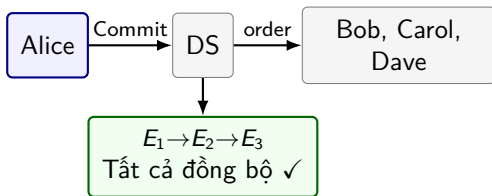
PCS: tự phục hồi sau lộ khóa.

Vấn đề: MLS cần thứ tự tuyến tính, P2P không có

MLS cần **Delivery Service** tuần tự hóa Commit. P2P không có DS → **phân nhánh trạng thái**.

MLS trên P2P (không DS)

MLS truyền thống (có DS)



Cùng $E+1$, $TH_1 \neq TH_2$
FORK!

► Một epoch = một Commit

Bài toán

MLS trên P2P thiếu Delivery Service → **phân nhánh trạng thái**. Cần cơ chế điều phối **nhẹ**, giữ nguyên bảo mật MLS, không sửa RFC 9420.

4 Mục tiêu

1. Giảm commits đồng thời
2. Xử lý trên epoch tương thích
3. Phát hiện & hàn gắn phân mảnh
4. Đảm bảo thứ tự hiển thị thông điệp ứng dụng

So sánh với các hướng tiếp cận hiện có

Tiêu chí	Quorum (Raft/BFT)	Blockchain	Giải pháp đề xuất
Mô hình nhất quán	Nhất quán mạnh	Nhất quán cuối cùng	Nhất quán cuối cùng
Chịu phân vùng	Kém (đóng băng nếu $< \frac{N}{2}$)	Tốt	Rất tốt (phân vùng nhỏ vẫn hoạt động)
Chi phí tạo Commit	$O(N) - O(N^2)$	Nặng (phát sóng khối)	$O(1)$ — 1 Token Holder
Độ trễ	Phụ thuộc RTT mạng	Rất cao (sinh khối)	Ngay lập tức (cục bộ)
Phù hợp P2P Chat	Kém	Không phù hợp	Rất cao

DMLS (draft-kohbrok): phức tạp, đánh đổi FS — **giải pháp đề xuất giữ nguyên FS/PCS của MLS**

1. Nghiên cứu

- ▶ 4 cơ chế điều phối (Single-Writer, epoch, fork healing, HLC)
- ▶ Không sửa lỗi MLS — giao thức bên ngoài RFC 9420
- ▶ Cơ sở lý thuyết CAP/SMR/FLP: chọn AP + Eventual Consistency
- ▶ Phân tích chi phí $O(1)$ / $O(\log N)$

2. Ứng dụng

- ▶ Ứng dụng desktop thử nghiệm (Wails + Go + Rust)
- ▶ 4 use-case: onboarding, chat, group management, file sharing
- ▶ Thử nghiệm RQ1–RQ5 trên mạng giả lập

Kết quả cốt lõi: giữ nguyên FS/PCS của MLS trong môi trường P2P không có DS

Giao thức điều phối đề xuất

Tại sao Single-Writer?

1. Tiến hóa nhóm = **SMR**: mỗi Commit là một chuyển trạng thái
2. Commit **không giao hoán**: thứ tự xử lý quyết định kết quả
3. **CAP**: P2P phân vùng $\Rightarrow$ chọn **AP** + **Eventual Consistency**; CP đóng băng, không phù hợp chat

Bầu chọn Token Holder xác định

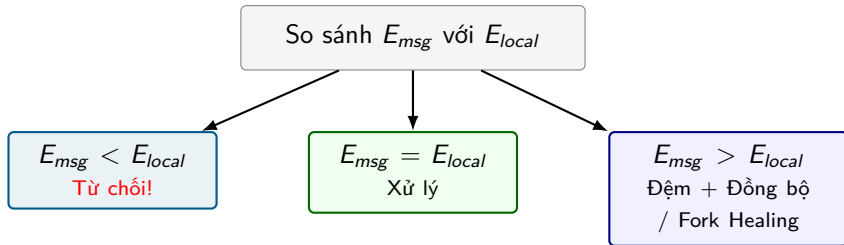
$$\text{Eligible}(E) = \text{Members}(E) \cap \text{ActiveView}(E) \setminus \text{Susp}(E)$$

$$\text{TokenHolder}(E) = \arg \min_{n \in \text{Eligible}(E)} H(\text{group_id} \| E \| n)$$

Cùng đầu vào $\rightarrow$ cùng kết quả | $O(N)$ cục bộ

- ▶ **Proposal**: mọi thành viên tạo
- ▶ **Commit**: chỉ Token Holder gom/phát
- ▶ **Failover**: ngoại tuyến $\rightarrow$ loại
ActiveView (Adaptive Lease +
 φ -Accrual — A)

Cơ chế 2: Kiểm tra epoch — Đơn điệu không giảm



- ▶ **Bất biến:** epoch cục bộ chỉ tăng hoặc giữ nguyên: $E_i(t+1) \geq E_i(t)$
- ▶ Thông điệp tương lai $\rightarrow$ đệm $\rightarrow$ kiểm tra tiền tổ lịch sử (HistoryHash)

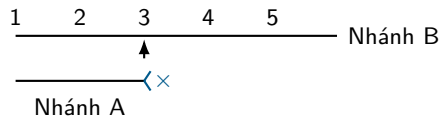
Cơ chế 3: Fork Detection — Phân biệt chậm vs phân nhánh

History Hash

$$R(0) = r_0, \quad R(e) = H(R(e-1) \| C(e))$$

$R(E)$ tóm tắt chuỗi Commit $1 \rightarrow E$

- ▶ $R_B(E_A) = R_A(E_A) \Rightarrow$ A chỉ chậm $\rightarrow$ catch up
- ▶ $R_B(E_A) \neq R_A(E_A) \Rightarrow$ **FORK thật** $\rightarrow$ Healing



$R_A(5) = R_B(5)?$
YES: chỉ chậm

NO: Fork thật

Hội tụ cần 3 điều kiện

(i) cùng epoch E , (ii) cùng TreeHash, (iii) cùng HistoryHash $R(E)$.

5 bước hàn gắn

1. Chọn nhánh thắng theo quy tắc xác định
2. Nút thua: hủy MLS cũ, tạo KeyPackage mới
3. Gửi External Proposal (Add) lên nhánh thắng
4. Nhánh thắng tạo **1 Commit + Welcome**
5. Các nút replay tin nhắn hợp lệ trên nhánh mới

Tại sao hiệu quả?

- ▶ **1 Commit** hàn gắn toàn bộ K nút lệch nhánh
- ▶ Tránh mỗi nút tự commit riêng lẻ $\rightarrow O(K^2)$ commits
- ▶ Crypto-shredding: hủy khóa nhánh thua, giữ FS/PCS
- ▶ Autonomous replay: chỉ re-send tin của chính mình



Vấn đề: Thứ tự hiển thị

- ▶ **2 loại tin:** Control (Proposal/Commit) vs Application (chat)
- ▶ **Có DS:** DS cấp timestamp
- ▶ **P2P:** clock drift → không tin physical clock

Tách 2 thứ tự: mật mã (epoch, Commit) $\neq$ hiển thị (HLC)

Giải pháp: Hybrid Logical Clock

$HLC = (L, C, \text{NodeID})$

$HLC_Now() :$

$L' = \max(L_{local}, L_{physical})$

if $L' = L_{local}$: $C++$ else $C=0$

return (L', C, NodeID)

- **Áp dụng:** application message
- **Đảm bảo:** happens-before + total order mỗi nút

Triển khai & Thực nghiệm

FRONTEND — React + TypeScript
Shadcn UI · Tailwind CSS · Zustand · Wails Bindings

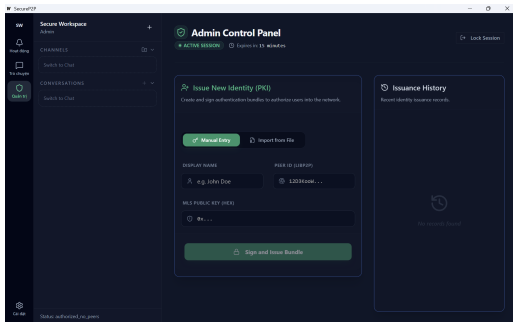
↓ Wails Bindings

TẦNG ĐIỀU PHỐI (Go) — Hexagonal Architecture
Single-Writer · Kiểm tra epoch · Fork Healing · HLC
libp2p+GossipSub · SQLite

↓ gRPC (stateless)

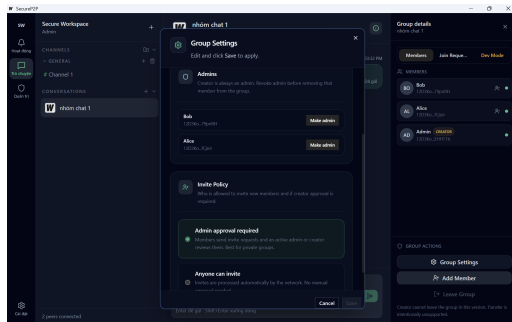
TẦNG MLS (Rust / OpenMLS) — Stateless Sidecar
Proposal · Commit · Welcome · TreeKEM · FS · PCS
Không biết về lớp điều phối — chỉ biến đổi mật mã

Ứng dụng desktop: Onboarding & Quản trị nhóm



1. Onboarding & Bundle

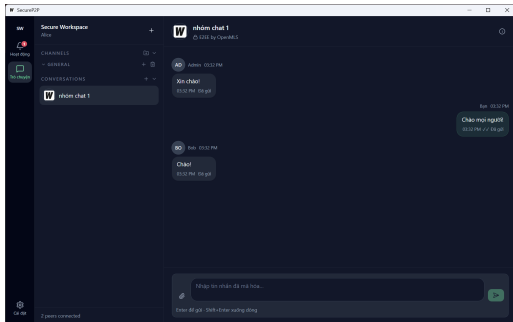
Thiết bị tự sinh khóa → QTV duyệt, cấp Bundle



2. Quản trị nhóm

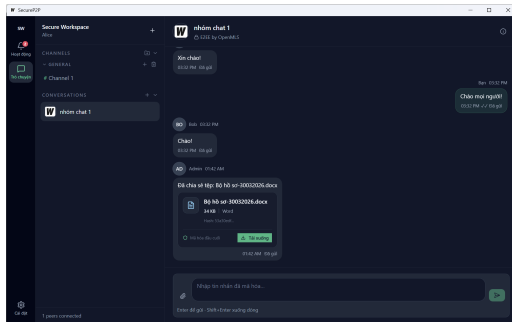
Xem thành viên, vai trò. Mỗi thao tác kích hoạt Proposal/Commit

Ứng dụng desktop: Chat nhóm & Chia sẻ tệp



3. Group Chat đồng bộ

Tin nhắn mã hóa MLS, hiển thị đồng bộ



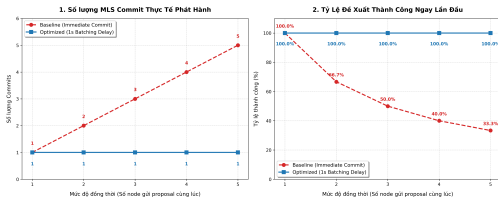
4. Chia sẻ tệp mã hóa

MLS exporter → AES-256-GCM. Khóa không truyền mạng

RQ1 + RQ2: Single-Writer giảm xung đột + Chaos test hội tụ

RQ1: Single-Writer có giảm xung đột?

ĐÁNH GIÁ HIỆU NĂNG PHÂN PHỐI ĐỀ XUẤT MLS (CONCURRENCY SWEEP EVALUATION)
So sánh Cơ chế Commit Ngay lập tức (Baseline) vs Gom Batch 1 Giây (Optimized)



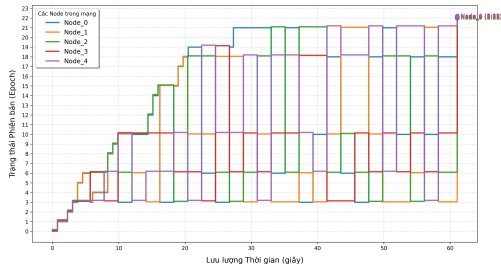
Xử lý ngay: tỷ lệ thành công 1.00 → 0.33

Single-Writer: tỷ lệ 1.00, Commit = 1

Cách đo: 5 nodes, mỗi node đề xuất 10 proposal; Single-Writer gom batch → 1 commit; Chaos test tạo partition 600ms lặp 5 lần.

RQ2: Hệ thống có hội tụ sau phân vùng?

Biểu đồ 3: Sự Hội Tụ Đồng Thuận - Khả Năng Khôi Phục (Fork Healing) P2P



5 nút, phân vùng lặp 600ms

Hội tụ: cùng epoch + tree hash

RQ3: Benchmark mật mã — MLS vs mã hóa từng cặp

Tạo nhóm 10–1000 thành viên; mã hóa tin nhắn 1 KB gửi toàn bộ nhóm; lặp 100 lần, lấy thời gian trung vị.

So sánh ở 4096 thành viên

Mốc đo	MLS (ms)	Từng cặp (ms)	Nhanh hơn
Trung vị	78,95	955,18	12,1 lần
p99	110,85	1081,72	9,8 lần

MLS giữ chi phí mã hóa ổn định khi nhóm lớn.

RQ4: Khả năng mở rộng — thao tác thành viên 5–1000 nút

Cách đo

Tạo sẵn nhóm gồm N nút. **1 nút** đề xuất Add/Remove/Update. Đo **tổng thời gian xử lý của toàn bộ N nút** để tất cả cùng tiến lên epoch tiếp theo.

Không tính độ trễ mạng.

Số trong bảng = chi phí tổng toàn mạng.

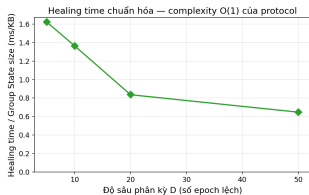
Nhận xét

VD: 1000 nút, Add: **2237 ms** tổng $\div 1000 \approx 2,2 \text{ ms/node}$

N	Thêm (ms)	Xóa (ms)	Cập nhật (ms)
5	0,00	0,52	0,00
10	0,53	1,05	0,52
50	6,19	7,26	6,59
100	24,53	23,53	24,67
250	130,47	129,70	128,83
500	527,36	531,18	517,97
750	1265,50	1169,21	1188,41
1000	2237,29	2194,79	2146,94

RQ5: Phục hồi phân nhánh + Chi phí điều phối + Forward Secrecy

RQ5a: Fork healing 1 External Commit cho mọi độ sâu D



D	Thời gian (ms)	State (KB)	ms/KB
5	728	449	1,62
10	1039	762	1,36
20	1155	1381	0,84
50	2098	3240	0,65

Nhánh sâu hơn thì group state lớn hơn, serialize nặng hơn nên tổng thời gian tăng. Nhưng ms/KB giảm cho thấy chi phí giao thức không phụ thuộc vào D , bản chất vẫn là $O(1)$.

RQ5b: Coordinator overhead

N	DP (ms)	Tổng (ms)	%
16	1,58	39,73	3,97
32	3,08	115,43	2,67
64	11,30	650,70	1,74

⇒ Giảm dần, luôn $< 4\%$

RQ5c: Forward Secrecy — thành viên bị loại không giải mã epoch mới ✓

Kết luận

Đóng góp chính

- ▶ **4 cơ chế điều phối** bao quanh MLS chuẩn (RFC 9420) — **không sửa lỗi MLS**
- ▶ **Fork Healing** $O(1)$: 1 External Commit, bất kể độ sâu phân kỳ
- ▶ **Eventual Consistency** phù hợp nhắn tin: AP, không đóng băng khi mất quorum
- ▶ **Ứng dụng desktop**: Wails + libp2p + OpenMLS thật, 5 nút thực tế

Giới hạn

- ▶ Chưa chứng minh formal
- ▶ Thực nghiệm trên mạng giả lập
- ▶ Group state truyền toàn bộ qua gRPC

Hướng phát triển

- ▶ Chứng minh formal giao thức
- ▶ Thử nghiệm multi-node vật lý
- ▶ Tối ưu giao tiếp Go $\leftrightarrow$ Rust

THÔNG TIN LIÊN HỆ

Sinh viên thực hiện: Lê Tiến Dũng – 20224837

Người hướng dẫn: TS. Đỗ Bá Lâm

- ▶ **Email:** dung.lt224837@sis.hust.edu.vn
- ▶ **Trường:** CNTT&TT – Đại học Bách khoa Hà Nội

Xin chân thành cảm ơn Hội đồng đã lắng nghe!

- [1] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara, and K. Cohn-Gordon, *The Messaging Layer Security (MLS) Protocol*, RFC 9420, IETF, Jul. 2023.
- [2] B. Beurdouche, E. Rescorla, E. Omara, S. Inguva, and A. Duric, *The Messaging Layer Security (MLS) Architecture*, RFC 9750, IETF, Apr. 2025.
- [3] J. Alwen, M. Mularczyk, and Y. Tselekounis, “Fork-Resilient Continuous Group Key Agreement,” in *Advances in Cryptology – CRYPTO 2023*, Springer, pp. 396–429, 2023.
- [4] K. Kohbrok, *Decentralized Messaging Layer Security*, Internet-Draft draft-kohbrok-mls-dmls-03, IETF, Oct. 2025.
- [5] F. B. Schneider, “Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial,” *ACM Computing Surveys*, vol. 22, no. 4, pp. 299–319, 1990.
- [6] S. Gilbert and N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.
- [7] W. Vogels, “Eventually Consistent,” *ACM Queue*, vol. 6, no. 6, pp. 14–19, 2008.



- [8] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” in *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pp. 305–319, 2014.
- [9] M. Castro and B. Liskov, “Practical Byzantine Fault Tolerance,” in *3rd Symposium on Operating Systems Design and Implementation (OSDI 99)*, pp. 173–186, 1999.
- [10] S. S. Kulkarni, M. Demirbas, D. Madeppa, B. Avva, and M. Leone, “Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases,” in *OPODIS 2014*, LNCS vol. 8878, Springer, pp. 17–32, 2014.
- [11] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” in *Proc. ACM SIGPLAN Onward! 2019*, 2019.
- [12] D. Vyzovitis, Y. Napora, D. McCormick, D. Dias, and Y. Psaras, “GossipSub: Attack-Resilient Message Propagation in the Filecoin and ETH2.0 Networks,” arXiv:2007.02754, 2020.
- [13] OpenMLS Project, *OpenMLS — A Rust Implementation of the Messaging Layer Security (MLS) Protocol*, 2026. <https://openmls.tech/>

- [14] Wails Project, *Wails — Build Desktop Applications using Go & Web Technologies*, 2026.
<https://wails.io/>

Vấn đề

Timeout cố định 4s dễ khiến nút suspend nhầm Token Holder khi GossipSub jitter, gây fork ngay cả khi mạng vẫn liên thông.

Adaptive Epoch Lease

$$L(E) = W_{batch} + \hat{C}_{commit} + 2\hat{RTT}_{p95}$$

Lease tính từ thời gian gom Proposal, tạo Commit và độ trễ p95 của mạng — thay thế timeout cố định.

φ -Accrual Failure Detector

$$\varphi(t) = -\log_{10}(1 - F_{HB}(t - t_{last}))$$

$\varphi = 1$: 10% còn sống (chậm)

$\varphi = 8$: 10^{-8} còn sống (chết)

Điều kiện suspend

$$\text{Suspend}(H) \iff L_{node}(E) \text{ hết} \wedge \varphi > 8$$

Chỉ suspend khi lease hết **và** xác suất holder chết cực cao.

Thời gian healing theo độ sâu phân kỳ

D	Healing (ms)	State (KB)	ms/KB
5	728	449	1,62
10	1039	762	1,36
20	1155	1381	0,84
50	2098	3240	0,65

Chi phí điều phối tuyệt đối

N	Điều phối (ms)	Tổng (ms)	Tỷ trọng (%)
16	1,58	39,73	3,97
32	3,08	115,43	2,67
64	11,30	650,70	1,74
128	35,55	—	—
256	134,86	—	—
512	539,69	—	—
1000	2204,85	—	—

A decorative graphic on the left side of the slide. It features a dark blue background with a large, stylized circular pattern composed of many small red dots. The dots are arranged in concentric, slightly offset rings, creating a sense of depth and movement. The word "HUST" is centered within this pattern.

HUST

THANK YOU !

